TWO SUM : UNIQUE PAIRS

Company: Amazon
Round: Online Assessment
Year: 2021
Difficulty: Easy
Topic: Arrays, HashSet / HashMap
Source: https://algo.monster/problems/two_sum_unique_pairs

Problem Description:

We are given:

- An array of numbers
- A target number

Our task is to find how many unique pairs of numbers add up to the target**.**

For example, if:

arr = [1, 1, 2, 45, 46, 46]
target = 47

The valid pairs are:

1 + 46 = 47
2 + 45 = 47

So the answer is : 2

Important Rule

The order of the numbers does not matter.

So :

[1, 5]

and :

[5, 1]

are considered the same pair.

Also, duplicate values should not create duplicate pairs.

Input Format:

The input contains:

- An array of integers
- A target integer

Output Format:

Return the number of unique pairs whose sum is equal to the target.

Sample Input:

arr[] = [1, 1, 2, 45, 46, 46]
target = 47

Sample Output:

Number of Unique Pairs = 2

Explanation:

Let's take the first example:

arr = [1, 1, 2, 45, 46, 46]
target = 47

We need two numbers whose sum is 47.

We go through the array one number at a time.

First 1

For 1, we need:

47 - 1 = 46

We haven't seen 46 yet, so we cannot form a pair.

We store 1.

Second 1

Again, we need 46.

But 46 still hasn't appeared, so we don't form a pair.

Since 1 is already stored, we don't need to store it again.

2

For 2, we need:

47 - 2 = 45

We haven't seen 45 yet, so we store 2.

45

Now we need:

47 - 45 = 2

We have already seen 2.

So we found:

2 + 45 = 47

Our count becomes 1.

First 46

For 46, we need:

47 - 46 = 1

We have already seen 1.

So we found:

1 + 46 = 47

Our count becomes 2.

Second 46

Again, we find the pair:

1 + 46 = 47

But we have already counted this pair.

So we don't count it again.

Therefore, the final answer is:

2

The two unique pairs are:

1 + 46 = 47
2 + 45 = 47

Why don't we count 46 + 1 separately?

Because the problem says:

[1, 46] = [46, 1]

They represent the same pair**.**

Similarly, if the array contains multiple 1s and 46s, we still count 1 + 46 only once.

In simple terms**:** we look for the number needed to reach the target, and whenever we find it, we count that pair only if we haven't counted the same pair before.

Approach to Solve This Problem:

The main idea is to use a HashSet**.**

A HashSet is useful because it stores values without duplicates**.**

As we go through the array, for every number x, we calculate the number we need:

needed = target - x

Then we check whether we have already seen that number.

Example

Suppose:

target = 47
x = 1

We need:

47 - 1 = 46

If 46 has already been seen, we have found a pair:

1 + 46 = 47

We add this pair to a set so that we don't count it again.

Why Do We Need a Set for Pairs?

Consider:

[1, 5, 1, 5]
target = 6

We may find:

1 + 5
5 + 1
1 + 5

But all of them represent the same pair:

[1, 5]

So we need to make sure that the same pair is counted only once.

We can store the smaller and larger value together as a string such as:

"1#5"

Then even if we find 5 + 1, we create the same key:

"1#5"

and the Set will not allow it to be counted twice.

Step-by-Step Example

Consider:

arr = [1, 1, 2, 45, 46, 46]
target = 47

We start with an empty set.

Number = 1

We need:

47 - 1 = 46

46 has not appeared yet.

Store 1.

Next Number = 1

Again, we need 46.

But we still haven't seen 46.

So we don't count anything.

Number = 2

We need:

47 - 2 = 45

45 has not appeared yet.

Store 2.

Number = 45

We need:

47 - 45 = 2

We have already seen 2.

So we found:

2 + 45 = 47

Count = 1.

Number = 46

We need:

47 - 46 = 1

We have already seen 1.

So we found:

1 + 46 = 47

Count = 2.

The next 46 creates the same pair again, so we ignore it. Final answer : 2

Java Code:

```java
import java.util.*;

public class Main {

    public static int countUniquePairs(int[] arr, int target) {

        HashSet<Integer> seen = new HashSet<>();
        HashSet<String> pairs = new HashSet<>();

        int count = 0;

        for (int num : arr) {

            int needed = target - num;

            // Check if the required number was already seen
            if (seen.contains(needed)) {

                // Store the pair in a fixed order
                int smaller = Math.min(num, needed);
                int larger = Math.max(num, needed);

                String pair = smaller + "#" + larger;

                // Count only if this pair is new
                if (pairs.add(pair)) {
                    count++;
                }
            }

            // Remember the current number
            seen.add(num);
        }

        return count;
    }
```

```java
public class Main {

    Run | Debug
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        // Number of elements
        int n = sc.nextInt();

        int[] arr = new int[n];

        // Input array
        for (int i = 0; i < n; i++) {
            arr[i] = sc.nextInt();
        }

        // Target value
        int target = sc.nextInt();

        System.out.println(countUniquePairs(arr, target));

        sc.close();
    }
}
```

Solution:

We use two HashSets:

- seen → stores numbers we have already encountered.
- pairs → stores the unique pairs we have already counted.

For every number:

1. Find the required number using target - num.
2. Check whether that number was already seen.
3. If yes, create a unique pair.
4. Add it to pairs.
5. Count it only if it is new.
6. Store the current number in seen.

This handles both duplicate values and reversed pairs correctly.

Understanding the Important Part of the Code:

1.seen HashSet

```
HashSet<Integer> seen = new HashSet<>();
```

This stores the numbers that we have already visited.

For example, after processing:

[1, 2, 45]

the set contains:

{1, 2, 45}

2. Find the Required Number

```
int needed = target - num;
```

This tells us what number we need to complete the pair.

For example:

target = 47
num = 2

Then:

needed = 47 - 2 = 45

So we look for 45.

3. Check if the Pair Exists

```
if (seen.contains(needed))
```

If the required number is already in seen, we have found a valid pair.

4. Create a Unique Pair

```
int smaller = Math.min(num, needed);
int larger = Math.max(num, needed);

String pair = smaller + "#" + larger;
```

We always put the smaller number first.

Therefore:

1 + 5
5 + 1

both become:

1#5

So they are treated as the same pair.

5. Count Only New Pairs

```
if (pairs.add(pair)) {
    count++;
}
```

HashSet.add() returns true only when the pair is new.

If the pair already exists, it returns false.

This prevents duplicate pairs from being counted.

6. Store the Current Number

```
seen.add(num);
```

After checking for a pair, we store the current number so that future elements can use it.

The order is important: we check first and add afterward.

Complexity Analysis:

Time Complexity: O(n)

We go through the array once.

HashSet operations such as contains() and add() take O(1) average time.

Therefore : Time Complexity = O(n)

Space Complexity: O(n)

In the worst case, we may store `n` different numbers and up to `n` unique pairs.

Therefore : Space Complexity = O(n)